TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC



BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Bài thực hành L06

Tối ưu hiệu năng Spark: shuffle, chiến lược join, data skew, AQE, phân vùng và cache

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 Mục tiêu và môi trường **2**

2 Thực hiện và kết quả **4**
2.1 E1: số partition shuffle và AQE coalescing 4
2.2 E2: broadcast hash join và sort-merge join 5
2.3 E3: data skew, salting và AQE skew join 8
2.4 E4: repartition và coalesce trước khi ghi 13
2.5 E5: cache 14
2.6 Bảng kết quả và độ ổn định 14

3 Trả lời câu hỏi checkpoint **19**
3.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh? 19
3.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết? 20
3.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng? 20
3.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu? 20
3.5 Khi nào không nên broadcast dù một phía "nhỏ"? 21
3.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước? 21
3.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không? . 21
3.8 Nên tinh chỉnh gì trước khi thêm phần cứng? 21

4 Khuyến nghị cho job production trên cluster 20 node **22**

5 Kết luận **22**

Tài liệu tham khảo **22**

Phụ lục: Trích đoạn mã nguồn **23**

Danh mục hình

1 Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle. 4
2 Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB. 5
3 Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join. 6
4 Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`. 7

5 Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API. 8
6 Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch. 9
7 Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s. 10
8 Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms. 11
9 Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`. 12
10 Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task. 13
11 Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ. 14
12 Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể. 15
13 Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai. 15
14 Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy. 18

Danh mục bảng

1 Môi trường thực nghiệm. 3
2 Dữ liệu thực nghiệm. 3
3 Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai). 8
4 Kết quả đầy đủ của lần chạy thứ hai (17 biến thể). 15
5 So sánh một số chỉ số giữa hai lần chạy. 18
6 So sánh với bảng kết quả tham chiếu của bài lab. 19
7 Danh mục tệp nộp kèm. 23

1 Mục tiêu và môi trường

Lab L06 đo tác dụng của các tham số tinh chỉnh Spark qua năm thí nghiệm: E1 số partition shuffle và cơ chế gộp partition của *Adaptive Query Execution* (AQE); E2 broadcast hash join so với sort-merge join; E3 data skew trong join, xử lý bằng salting và AQE skew join; E4 `repartition` so với `coalesce` trước khi ghi; E5 khi nào `cache`/`persist` có lợi. Mọi thứ chạy trong một container Docker ở chế độ local 4 core (Bảng 1).

Bảng 1. Môi trường thực nghiệm.

Thành phần	Giá trị
Máy chủ	Laptop 12 luồng CPU, 32 GB RAM
Ảo hóa	Docker Desktop trên WSL2, cấp 31 GB
Spark	4.1.3 (image `spark:4.1.3-python3`), PySpark
Chế độ chạy	`local[4]` (4 core, `defaultParallelism` = 4), driver 2 GB
Thu thập số liệu	`time.time()` quanh mỗi action; Spark REST API (`/sql`, `/stages/.../taskSummary`)
Ngày thực hiện	2026-10-30

Dữ liệu là tập chuyến taxi New York tháng 01/2024 và hai bảng tổng hợp cho E3 (Bảng 2).

Bảng 2. Dữ liệu thực nghiệm.

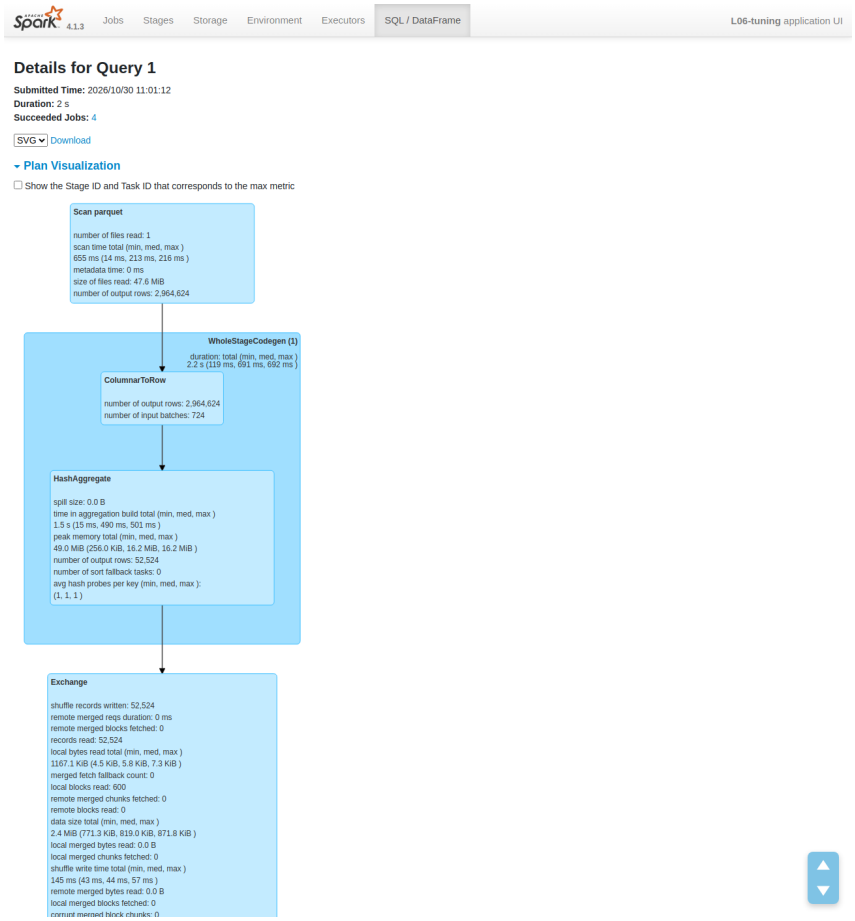
Tập dữ liệu	Nguồn	Kích thước	Dùng trong
Trips	`yellow_tripdata_2024-01.parquet`	2,964,624 dòng, 47.6 MiB, 3 row group	E1, E2, E4, E5
Zones	`taxi_zone_lookup.csv`	265 dòng	E2
Fact tổng hợp	`spark.range(10,000,000)`, `HOT_SHARE = 0.6` (key = 0), còn lại rải đều trên 200,000 khóa	10,000,000 dòng (5,998,366 hot, 4,001,634 thường)	E3
Dim tổng hợp	`spark.range(200,000)`, cột `segment` = `seg_{key % 50}`	200,000 dòng	E3

Script `l06_experiments.py` chạy một `count()` khởi động không tính giờ, rồi với mỗi biến thể thì đặt lại cấu hình (`reset_conf`), đổi tham số và ép thực thi bằng `df.write.format("noop")` (tính đủ nhưng không ghi file); E4 ghi Parquet thật để đếm file, E5 dùng `collect()`. Sau mỗi action, hàm `last_sql_summary()` gọi REST API [2] lấy các nút của kế hoạch cuối (sau AQE), tổng số task, và median/max thời gian task ở stage lệch nhất (stage có hiệu max trừ median lớn nhất). Chúng tôi chạy cả script hai lần (34 dòng trong `results.csv`) và dùng lần thứ hai; cờ `--hold` giữ Spark UI để chụp kế hoạch và Summary Metrics.

2 Thực hiện và kết quả

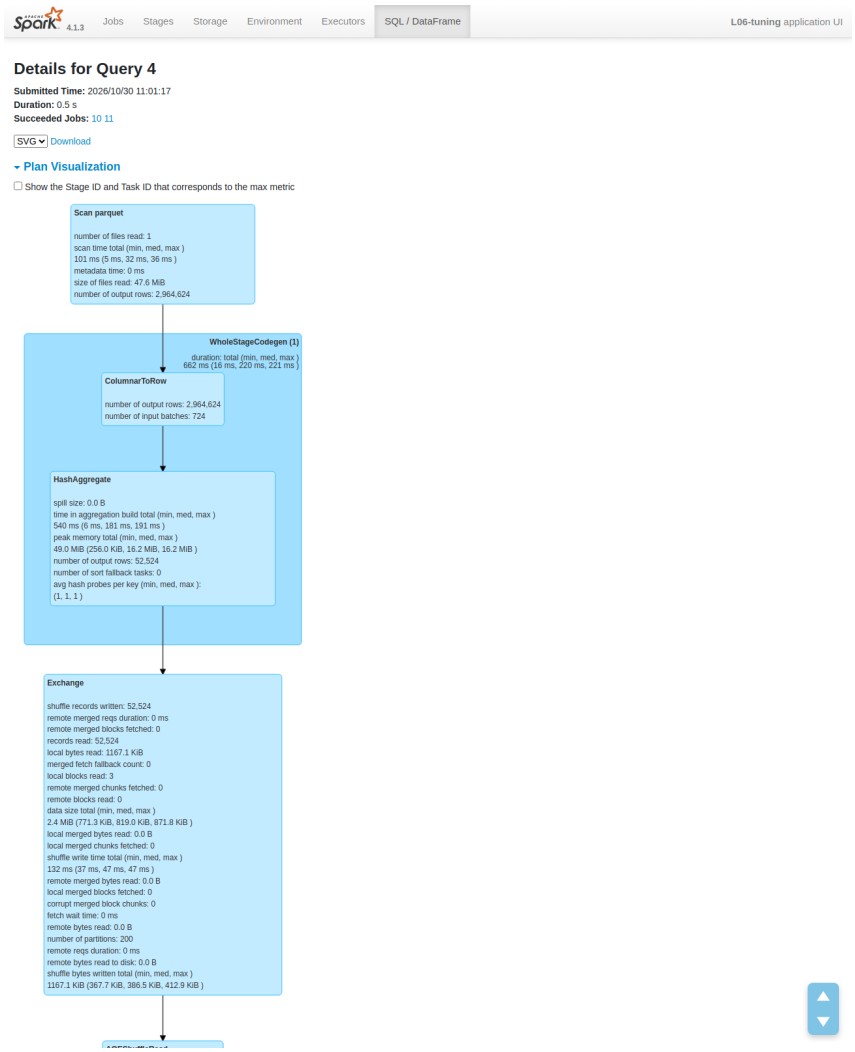
2.1 E1: số partition shuffle và AQE coalescing

Truy vấn `groupBy(PULocationID, DOLocationID).agg(count, avg)` đọc 47.6 MiB nhưng chỉ ghi khoảng 1167.1 KiB dữ liệu shuffle. Script chạy nó với AQE tắt và `shuffle.partitions` $\in \{200, 8, 1\}$; phần chúng tôi tự viết là biến thể bật AQE, giữ 200 partition. Với 200 partition và AQE tắt, truy vấn sinh 204 task (4 map, 200 reduce), mỗi task reduce chạy khoảng 6 ms, chậm nhất trong E1; kế hoạch chỉ có một `Exchange` 200 partition (Hình 1).



Hình 1. Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle.

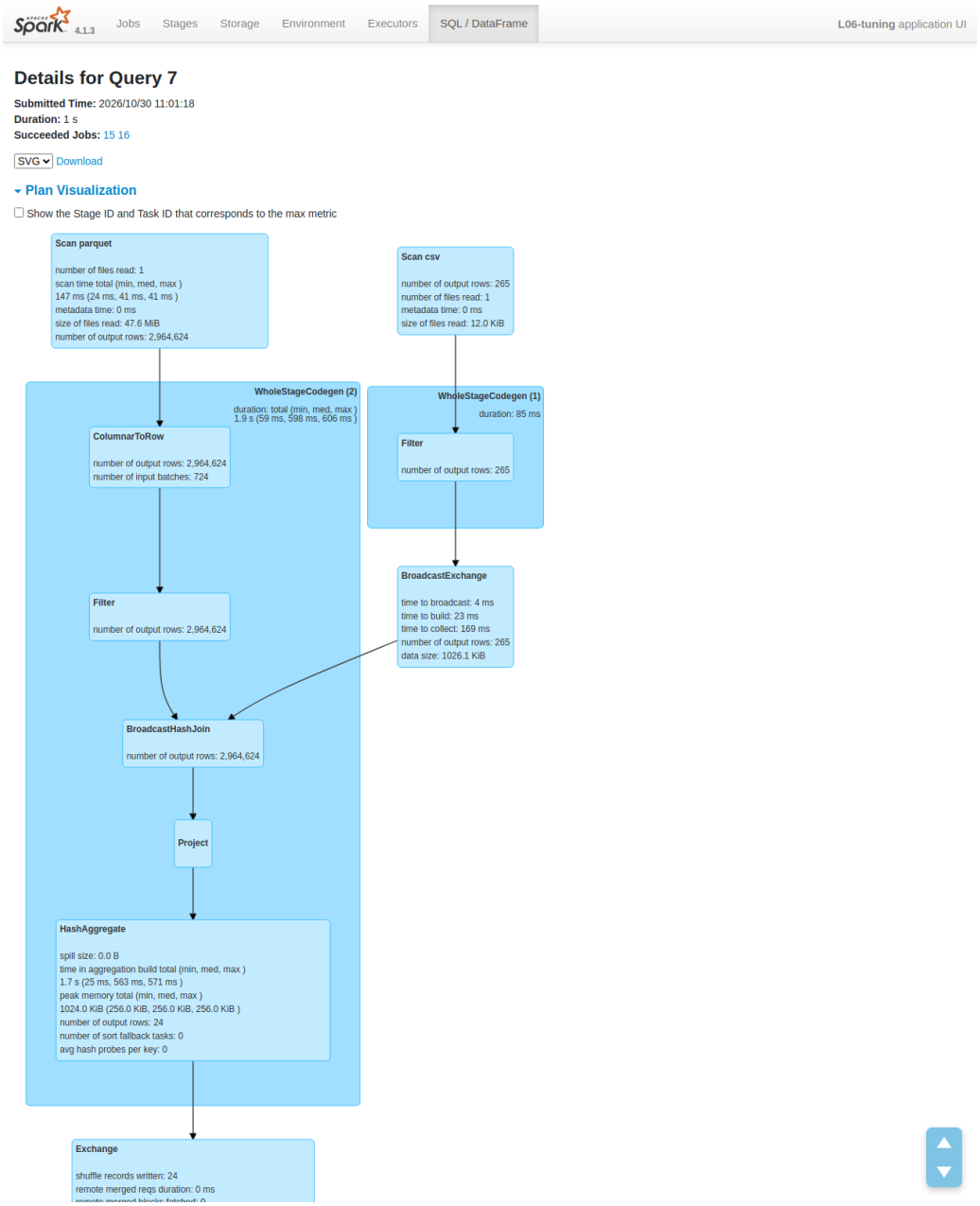
AQE tối ưu lại kế hoạch trong lúc chạy, dựa trên thống kê thật của stage shuffle đã xong thay vì ước lượng [1]. Bật AQE mà vẫn để 200 partition thì sau `Exchange` xuất hiện `AQEShuffleRead` (Hình 2), các partition nhỏ được gộp lại, tổng chỉ còn 5 task, thời gian xấp xỉ bản tinh chỉnh tay 8 partition (Bảng 4). Bản 1 partition cũng nhanh, nhưng chỉ vì kết quả quá nhỏ (khoảng 20 nghìn nhóm), xem Mục 3.1.



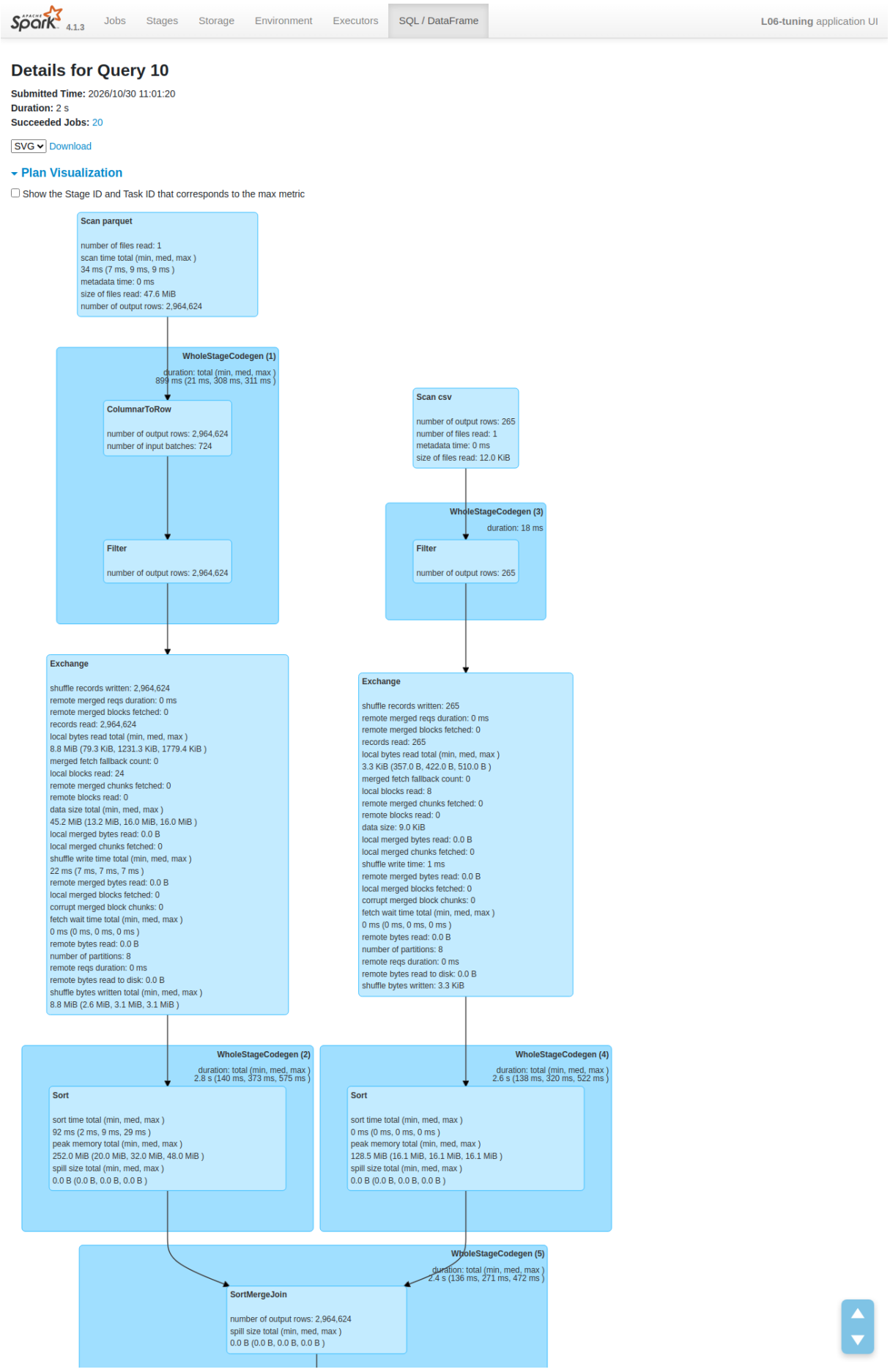
Hình 2. Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB.

2.2 E2: broadcast hash join và sort-merge join

Phép join trips ⋈ zones được chạy với `autoBroadcastJoinThreshold` 10 MB, với ngưỡng -1, và (phần tự viết) với ngưỡng -1 kèm gợi ý `F.broadcast(zones_df())`. Khi ngưỡng là -1, Spark phải dùng `SortMergeJoin`: khoảng 3 triệu chuyến bị shuffle cùng bảng zone rồi sort ở cả hai phía (Hình 4). Khi broadcast, chỉ 265 dòng được gửi tới mọi executor và phía trips không có `Exchange` trước join (Hình 3).



Hình 3. Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join.



Hình 4. Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`.

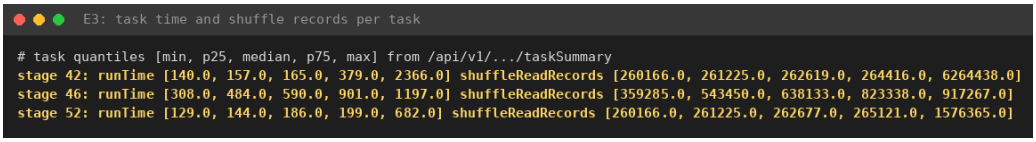
Đặt ngưỡng -1 chỉ tắt việc chọn broadcast tự động của planner [3]; gợi ý `broadcast()` vẫn có hiệu lực, cho kế hoạch `BroadcastHashJoin` và là biến thể nhanh nhất của E2 (0.40 s, 6 task). Hơi bất ngờ là broadcast tắt AQE (1.20 s) chỉ nhanh hơn SMJ (1.51 s) một chút, có lẽ vì script mặc định chỉ dùng 8 partition nên SMJ ở đây khá nhẹ, cộng thêm dao động giữa các lần chạy (Bảng 5). Khác biệt thật nằm ở kế hoạch: broadcast bỏ được `Exchange` và `Sort` ở phía bảng lớn, và khoản này sẽ lớn dần theo kích thước dữ liệu.

2.3 E3: data skew, salting và AQE skew join

Shuffle băm theo khóa nên mọi bản ghi của khóa 0 (60 % bảng fact) dồn vào một partition. Ba biến thể được chạy: sort-merge join với AQE tắt, salting với 64 giá trị salt, và AQE skew join. Phân vị thời gian task và số bản ghi shuffle đọc mỗi task tại stage join của từng biến thể được lấy qua REST API (Bảng 3, Hình 5).

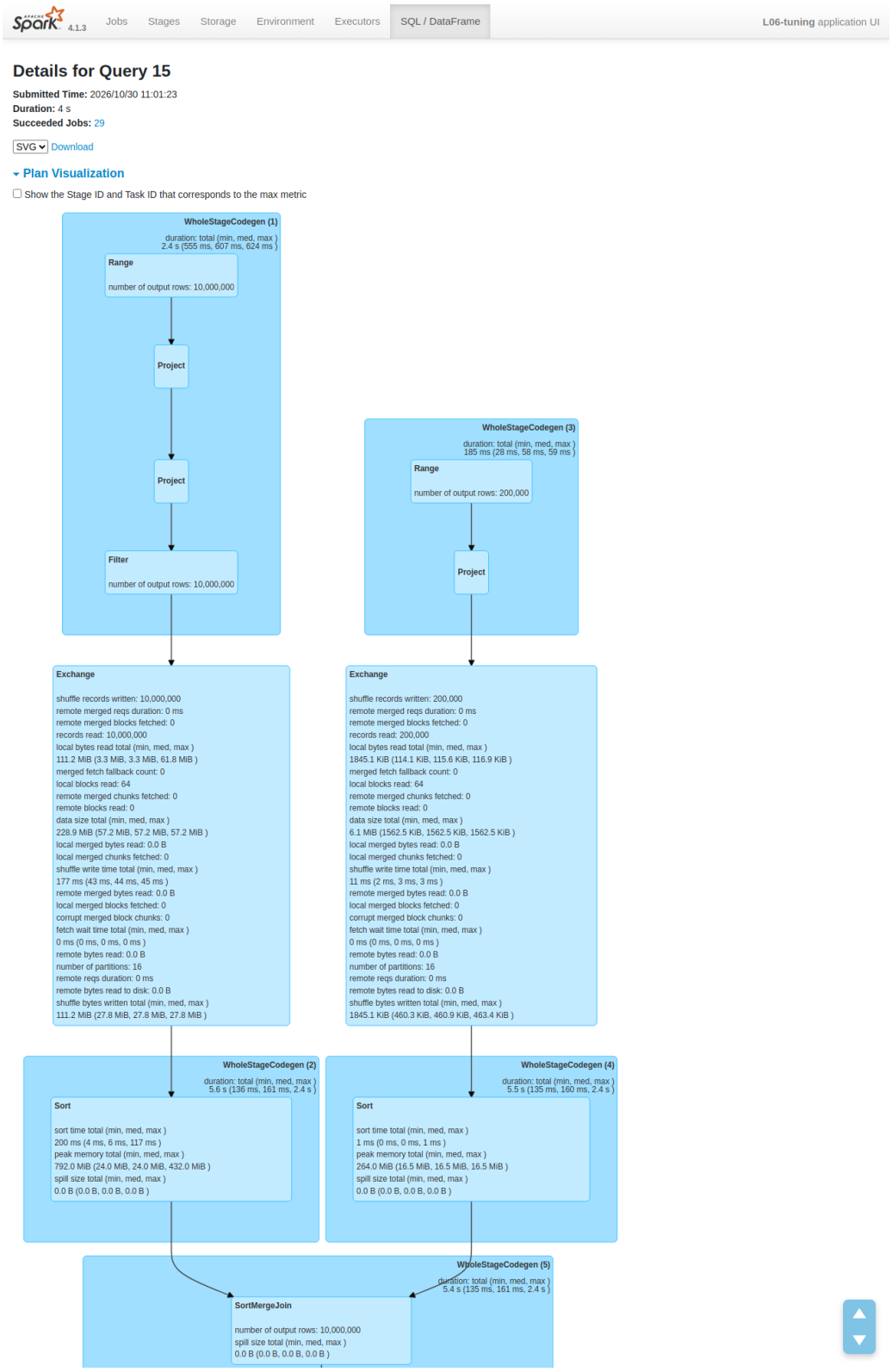
Bảng 3. Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai).

Stage (biến thể)	Thời gian task (ms)	Bản ghi shuffle đọc / task
42: baseline, AQE off	140 / 157 / 165 / 379 / 2,366	260,166 / 261,225 / 262,619 / 264,416 / 6,264,438
46: salting 64	308 / 484 / 590 / 901 / 1,197	359,285 / 543,450 / 638,133 / 823,338 / 917,267
52: AQE skew join	129 / 144 / 186 / 199 / 682	260,166 / 261,225 / 262,677 / 265,121 / 1,576,365

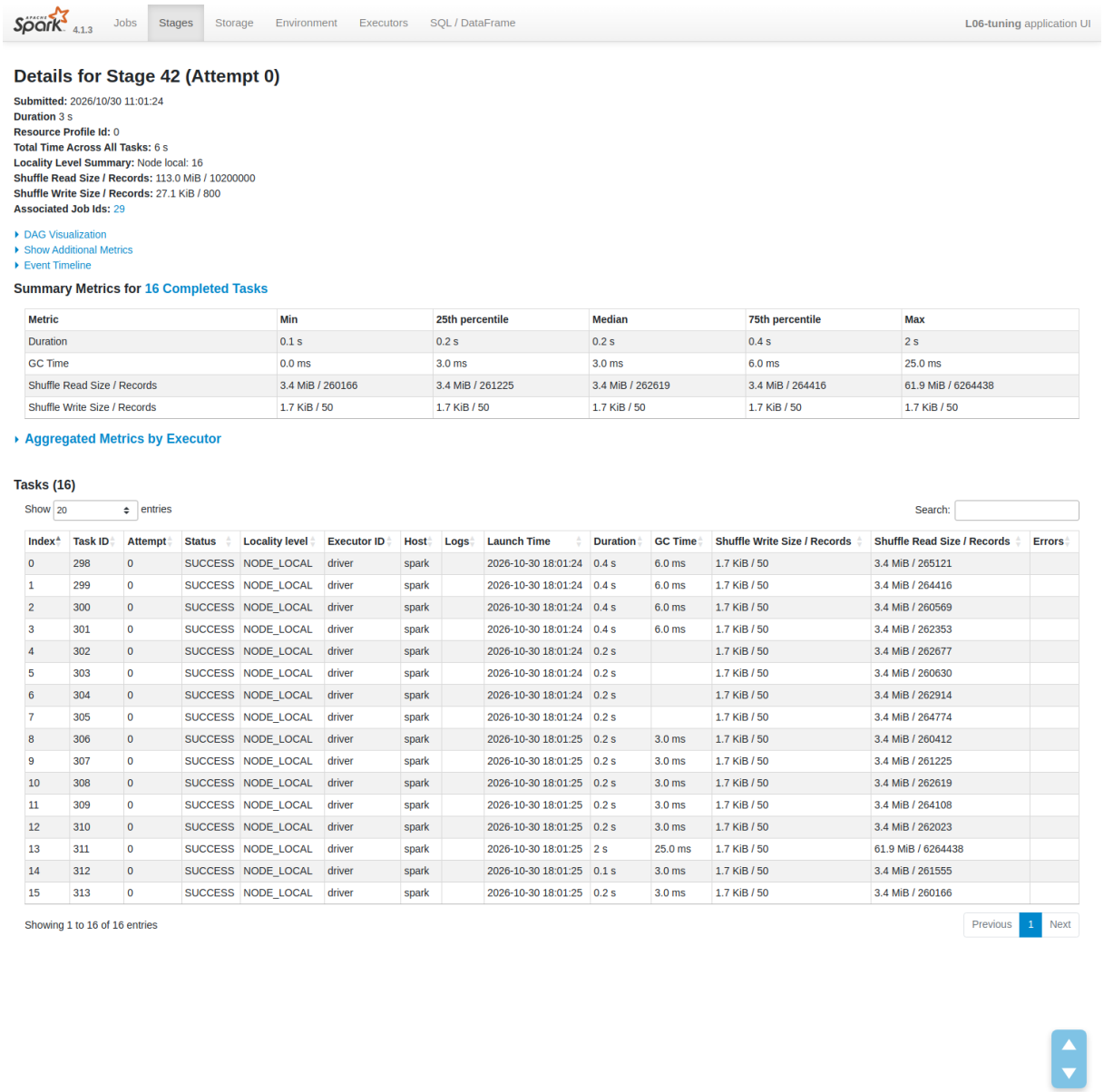


Hình 5. Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API.

Baseline. Kế hoạch là `Exchange` + `SortMergeJoin` với 16 partition (Hình 6). Ở stage 42, task chậm nhất lâu hơn median khoảng 14 lần. Theo Summary Metrics (Hình 7), task này đọc 61.9 MiB, trong khi các task khác chỉ khoảng 3.4 MiB. Cả stage đọc 113.0 MiB / 10,200,000 bản ghi qua 16 task. Trong lúc task đó chạy, ba core còn lại không có việc.



Hình 6. Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 42 (Attempt 0)

Submitted: 2026/10/30 11:01:24
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 6 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 113.0 MiB / 10200000
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 29

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.2 s	0.2 s	0.4 s	2 s
GC Time	0.0 ms	3.0 ms	3.0 ms	6.0 ms	25.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262619	3.4 MiB / 264416	61.9 MiB / 6264438
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (16)

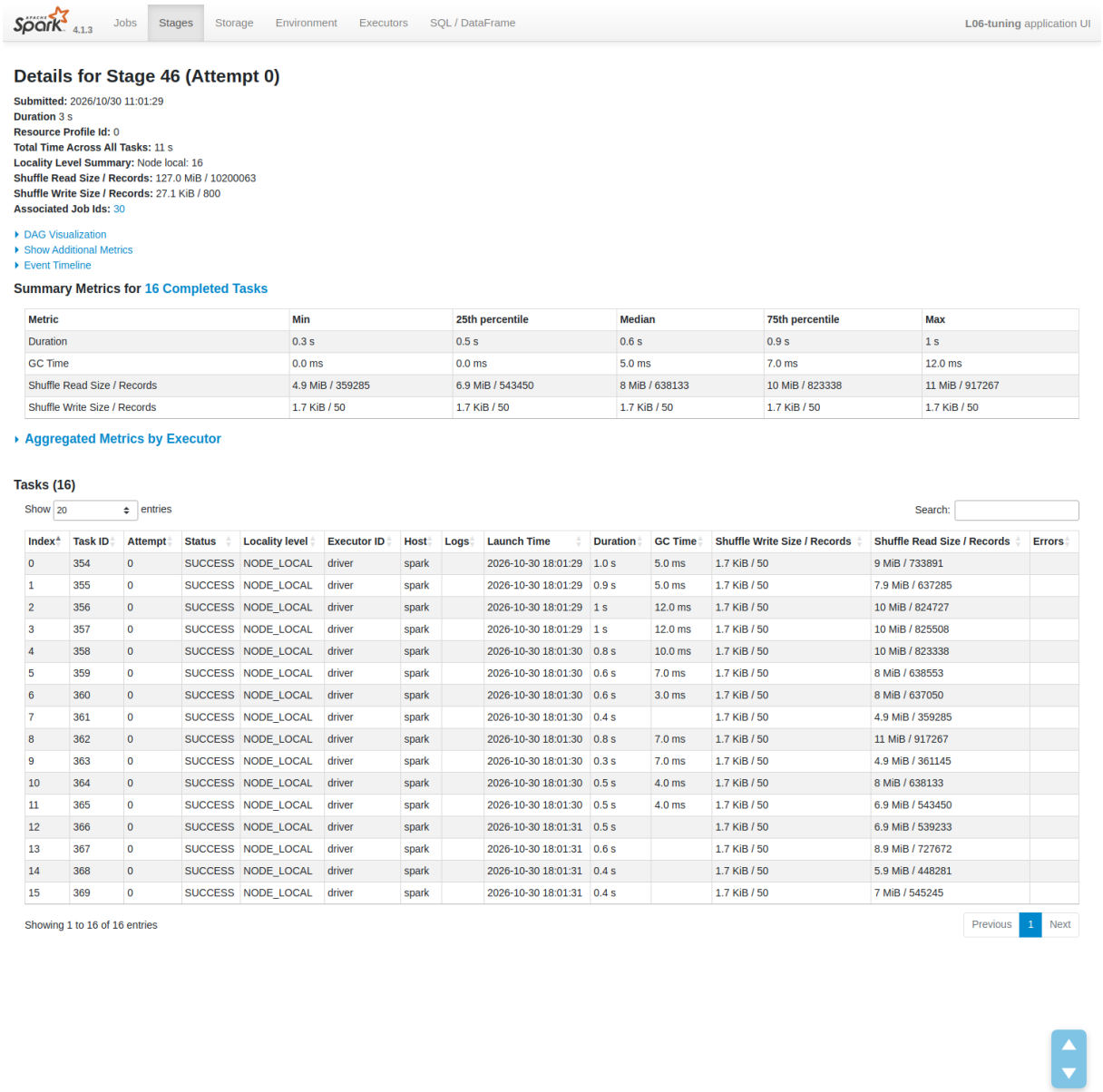
Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	298	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 265121	
1	299	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 264416	
2	300	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 260569	
3	301	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 262353	
4	302	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	303	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	304	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	305	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	306	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260412	
9	307	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261225	
10	308	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262619	
11	309	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 264108	
12	310	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262023	
13	311	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	2 s	25.0 ms	1.7 KiB / 50	61.9 MiB / 6264438	
14	312	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.1 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
15	313	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 16 of 16 entries Previous 1 Next

Hình 7. Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s.

Salting. Chỉ hot key được gắn salt ngẫu nhiên 0..63, dòng dim tương ứng được nhân lên 64 bản, rồi join theo `(key, salt)`. Task max giảm gần một nửa và số bản ghi lớn nhất mỗi task chỉ còn chưa tới một triệu (Hình 8). Bù lại, median tăng lên 590 ms (lý do ở Mục 3.3) và thời gian tổng gần như đứng yên (4.01 s so với 4.10 s). Tổng `amount` vẫn đúng (500061629), nên salting không làm sai kết quả.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 46 (Attempt 0)

Submitted: 2026/10/30 11:01:29
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 11 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 127.0 MiB / 10200063
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 30

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.3 s	0.5 s	0.6 s	0.9 s	1 s
GC Time	0.0 ms	0.0 ms	5.0 ms	7.0 ms	12.0 ms
Shuffle Read Size / Records	4.9 MiB / 359285	6.9 MiB / 543450	8 MiB / 638133	10 MiB / 823338	11 MiB / 917267
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (16)

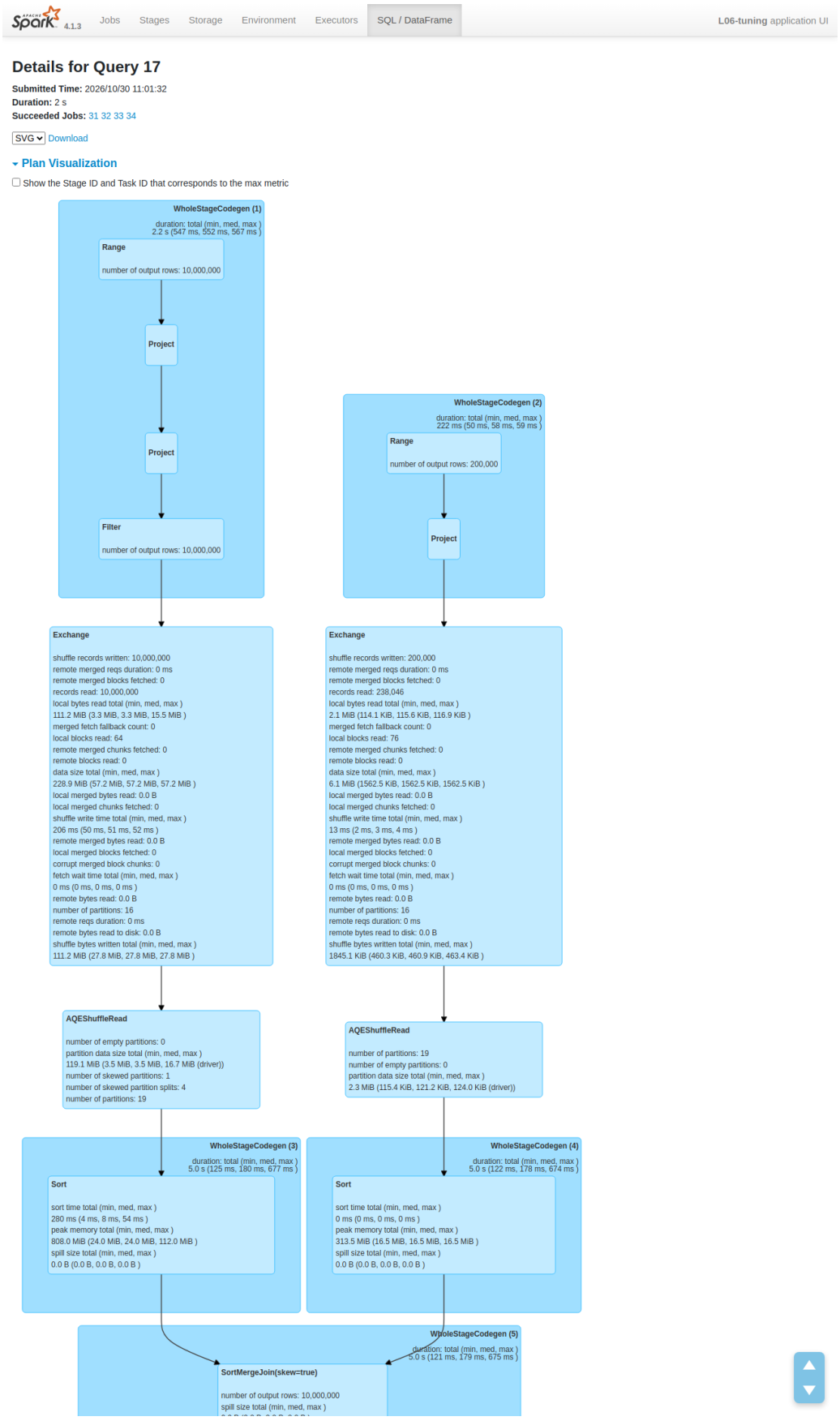
Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	354	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1.0 s	5.0 ms	1.7 KiB / 50	9 MiB / 733891	
1	355	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	0.9 s	5.0 ms	1.7 KiB / 50	7.9 MiB / 637285	
2	356	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 824727	
3	357	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 825508	
4	358	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	10.0 ms	1.7 KiB / 50	10 MiB / 823338	
5	359	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	7.0 ms	1.7 KiB / 50	8 MiB / 638553	
6	360	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	3.0 ms	1.7 KiB / 50	8 MiB / 637050	
7	361	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.4 s		1.7 KiB / 50	4.9 MiB / 359285	
8	362	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	7.0 ms	1.7 KiB / 50	11 MiB / 917267	
9	363	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.3 s	7.0 ms	1.7 KiB / 50	4.9 MiB / 361145	
10	364	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	8 MiB / 638133	
11	365	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	6.9 MiB / 543450	
12	366	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.5 s		1.7 KiB / 50	6.9 MiB / 539233	
13	367	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.6 s		1.7 KiB / 50	8.9 MiB / 727672	
14	368	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	5.9 MiB / 448281	
15	369	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	7 MiB / 545245	

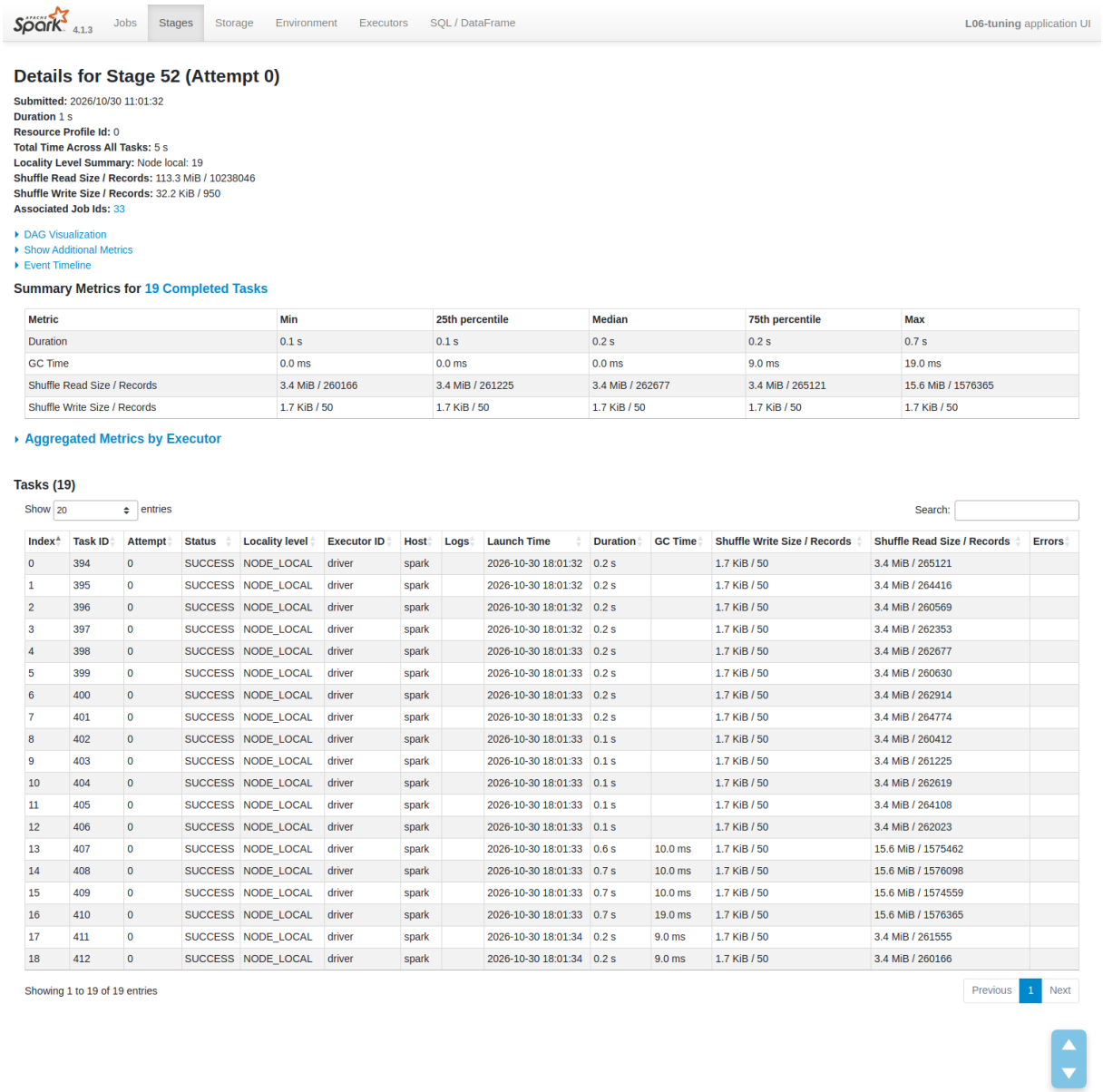
Showing 1 to 16 of 16 entries Previous 1 Next

Hình 8. Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms.

AQE skew join. Cấu hình bật AQE và skew join, đặt `advisoryPartitionSizeInBytes=4m` , `skewedPartitionThresholdInBytes=16m`, `skewedPartitionFactor=3` và tắt coalesce. Kế hoạch cuối có `SortMergeJoin(skew=true)`, và `AQEShuffleRead` báo *number of skewed partitions: 1, skewed partition splits: 4, number of partitions: 19* (Hình 9). Phía fact ghi 10,000,000 bản ghi shuffle (111.2 MiB); sau khi đọc, partition có median 3.5 MiB và max 16.7 MiB. Ở stage 52, số bản ghi lớn nhất mỗi task xấp xỉ một phần tư của baseline, đúng với 4 split (Hình 10). Thời gian tổng còn **2.22 s, nhanh gần gấp đôi baseline**, mà không phải sửa dòng truy vấn nào. Lúc đầu chúng tôi để ngưỡng mặc định thì AQE không tách gì, vì partition nóng chỉ có 61.9 MiB, thấp hơn nhiều so với mức 256 MB mặc định, nên phải hạ ngưỡng như trên.



Hình 9. Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 52 (Attempt 0)

Submitted: 2026/10/30 11:01:32
Duration 1 s
Resource Profile Id: 0
Total Time Across All Tasks: 5 s
Locality Level Summary: Node local: 19
Shuffle Read Size / Records: 113.3 MiB / 10238046
Shuffle Write Size / Records: 32.2 KiB / 950
Associated Job Ids: 33

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 19 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.1 s	0.2 s	0.2 s	0.7 s
GC Time	0.0 ms	0.0 ms	0.0 ms	9.0 ms	19.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262677	3.4 MiB / 265121	15.6 MiB / 1576365
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (19)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	394	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 265121	
1	395	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 264416	
2	396	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 260569	
3	397	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 262353	
4	398	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	399	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	400	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	401	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	402	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 260412	
9	403	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 261225	
10	404	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262619	
11	405	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 264108	
12	406	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262023	
13	407	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.6 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1575462	
14	408	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1576098	
15	409	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1574559	
16	410	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	19.0 ms	1.7 KiB / 50	15.6 MiB / 1576365	
17	411	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
18	412	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 19 of 19 entries Previous 1 Next

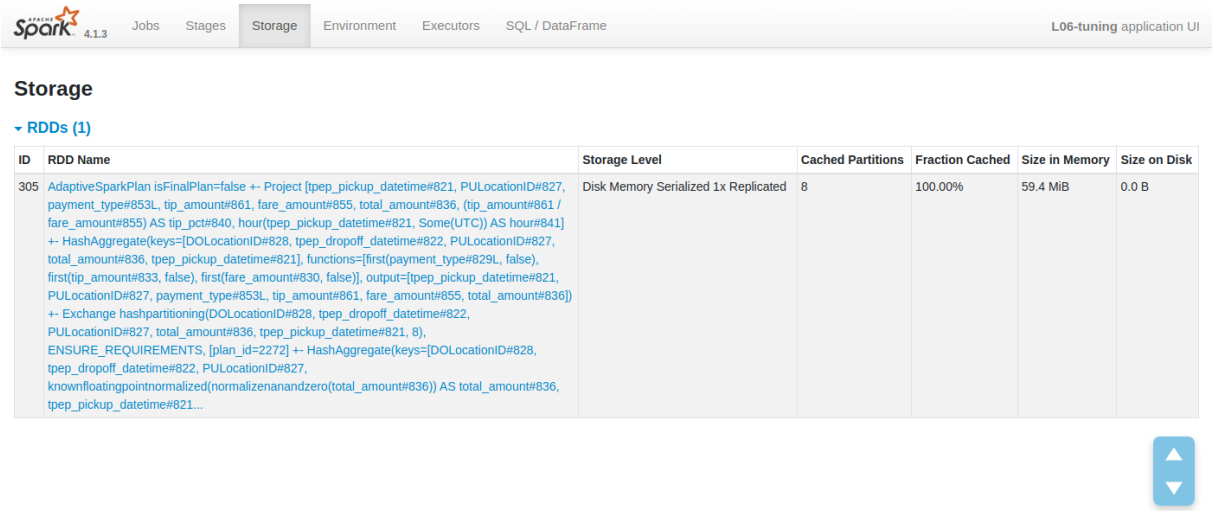
Hình 10. Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task.

2.4 E4: repartition và coalesce trước khi ghi

Trips được ghi ra Parquet theo bốn cách: `repartition(64)`, `repartition(4)`, `coalesce(4)`, `coalesce(1)`. `repartition(64)` chậm nhất vì vừa có một shuffle đầy đủ vừa ghi 64 file nhỏ. `coalesce(4)` không shuffle nên nhanh nhất, nhưng chỉ ra 3 file chứ không phải 4. Ban đầu chúng tôi nghĩ có lỗi; xem lại thì file Parquet đầu vào chỉ có 3 row group nên một trong 4 partition đầu vào rỗng. `coalesce(1)` dồn cả đọc lẫn ghi vào một task và mất hết song song, còn `repartition(4)` cho 4 file đều nhau với giá một shuffle (Bảng 4).

2.5 E5: cache

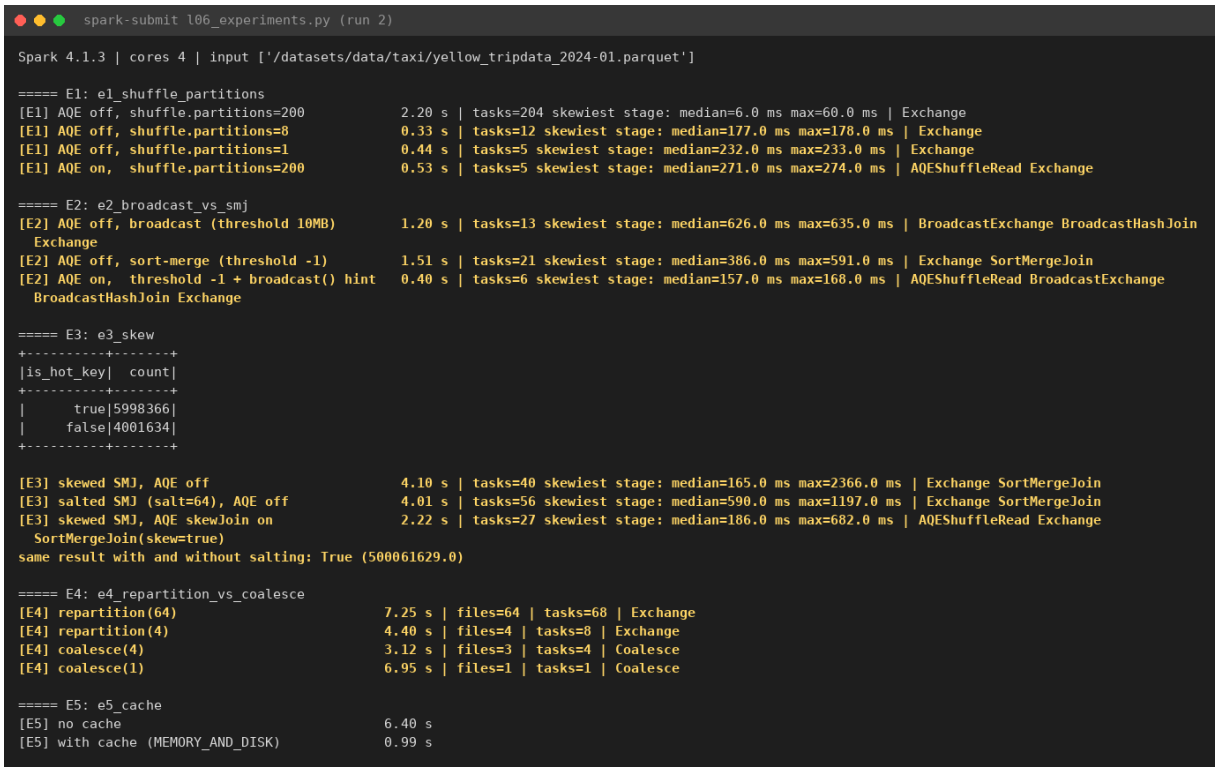
DataFrame "cleaned" gồm một phép lọc và `dropDuplicates` trên năm cột, tức là shuffle toàn bộ khoảng 3 triệu dòng. Nó được dùng cho ba phép tổng hợp (theo giờ, theo `PULocationID`, theo `payment_type`), lần đầu không cache, lần sau với `persist(MEMORY_AND_DISK)`. Không cache thì dedup chạy lại ba lần. Có cache thì `count()` để materialize mất 3.42 s và trả về 2,869,714 dòng, sau đó ba truy vấn chỉ còn **0.99 s**. Tab Storage (Hình 11) cho thấy 8/8 partition đã được cache (100 %), chiếm 59.4 MiB bộ nhớ và 0.0 B đĩa, mức `Disk Memory Serialized 1x Replicated`; REST API báo `memoryUsed` = 62,240,847 byte. Tính cả thời gian materialize, cache vẫn lời (khoảng 4.4 s so với 6.4 s).



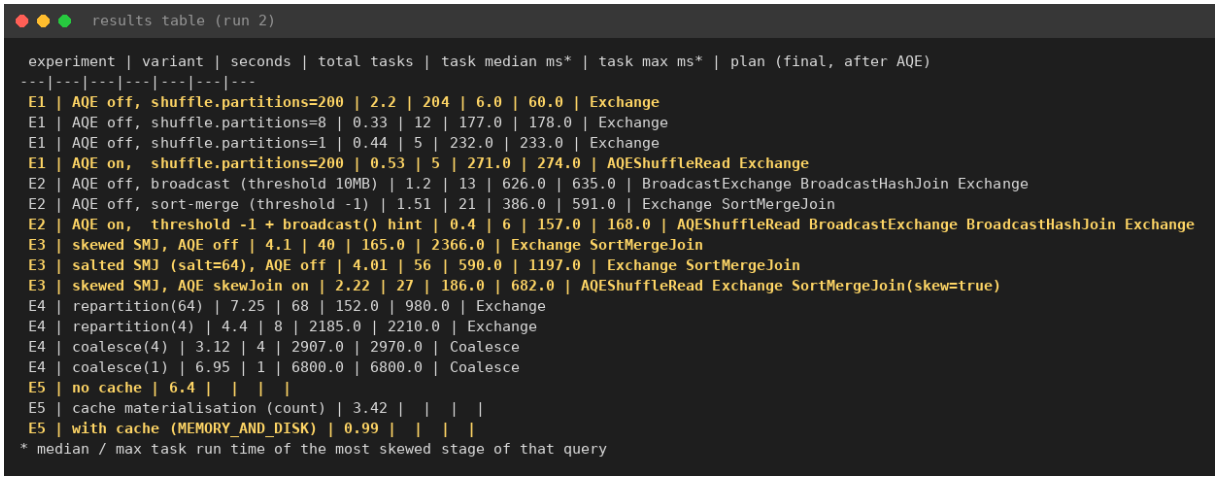
Hình 11. Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ.

2.6 Bảng kết quả và độ ổn định

Theo đề, bảng ghi nhận là của lần chạy thứ hai. Hình 12 là đầu ra console của lần chạy thứ hai, Hình 13 là bảng Markdown script in ra ở cuối, và Bảng 4 gom đủ 17 biến thể; hai cột median/max là thời gian task ở stage lệch nhất. Tab SQL của phiên có giữ UI liệt kê 31 truy vấn đã xong (Hình 14), chúng tôi dùng danh sách này để tìm kế hoạch của từng biến thể.



Hình 12. Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể.



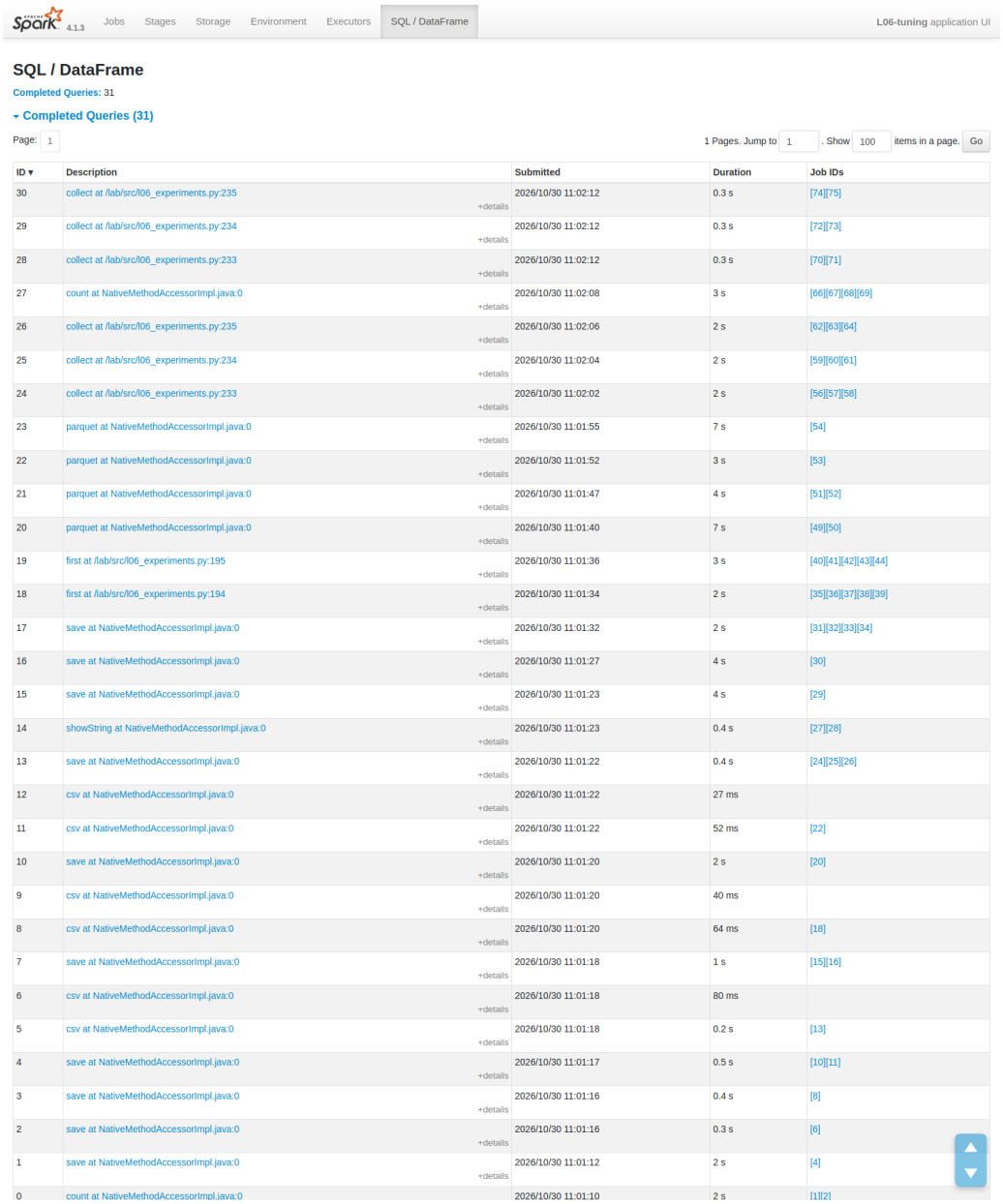
Hình 13. Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai.

Bảng 4. Kết quả đầy đủ của lần chạy thứ hai (17 biến thể).

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E1	AQE off, shuffle.partitions=200	2.20	204	6	60	Exchange	

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E1	AQE off, shuffle.partitions=8	0.33	12	177	178	Exchange	
E1	AQE off, shuffle.partitions=1	0.44	5	232	233	Exchange	
E1	AQE on, shuffle.partitions=200	0.53	5	271	274	AQEShuffleRead Exchange	AQE gộp partition nhỏ
E2	AQE off, broadcast (threshold 10MB)	1.20	13	626	635	BroadcastExchange BroadcastHashJoin Exchange	
E2	AQE off, sort-merge (threshold -1)	1.51	21	386	591	Exchange SortMergeJoin	
E2	AQE on, threshold -1 + broadcast() hint	0.40	6	157	168	AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange	Không đổi kế hoạch dù threshold = -1
E3	skewed SMJ, AQE off	4.10	40	165	**2366**	Exchange SortMergeJoin	
E3	salted SMJ (salt=64), AQE off	4.01	56	590	1197	Exchange SortMergeJoin	cùng kết quả (500061629)
E3	skewed SMJ, AQE skewJoin on	2.22	27	186	682	AQEShuffleRead Exchange SortMergeJoin(skew=true)	1 partition skew, 4 split
E4	repartition(64)	7.25	68	152	980	Exchange	64 file
E4	repartition(4)	4.40	8	2185	2210	Exchange	4 file
E4	coalesce(4)	3.12	4	2907	2970	Coalesce	3 file
E4	coalesce(1)	6.95	1	6800	6800	Coalesce	1 file

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E5	no cache (3 queries)	6.40					3 aggregation
E5	cache materialisation (count)	3.42					2,869,714 dòng, 59.4 MiB
E5	with cache (3 queries)	0.99					3 aggregation



Hình 14. Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy.

Lần chạy đầu (tổng 71.71 s) cho cùng xu hướng ở mọi thí nghiệm, phần lớn chênh lệch nằm trong khoảng ±20 % mà đề bài cho phép (Bảng 5). Ngoại lệ là các biến thể rất ngắn như E2 broadcast tắt AQE và E1 một partition: với truy vấn dưới một giây, vài trăm mili giây dao động đã là tỉ lệ lớn, nên chúng tôi không rút kết luận từ chênh lệch nhỏ giữa các biến thể loại này. Vì chạy local một máy, không có chi phí mạng, cột max/median đáng tin hơn thời gian tuyệt đối.

Bảng 5. So sánh một số chỉ số giữa hai lần chạy.

Chỉ số	Lần 1	Lần 2
E1, 200 partition, AQE off	2.15 s	2.20 s
E1, 1 partition, AQE off	0.30 s	0.44 s

Chỉ số	Lần 1	Lần 2
E2, broadcast, AQE off	0.77 s	1.20 s
E3 baseline, task max	2,686 ms	2,366 ms
E3 salting	4.28 s	4.01 s
E3 AQE skew join	2.28 s	2.22 s
E5 không cache	6.07 s	6.40 s
E5 với cache	0.88 s	0.99 s

So với bảng tham chiếu của đề (Apple M-series, Docker 4 GB, `local[4]`), thời gian trên máy chúng tôi cao hơn khoảng 1.3 đến 1.9 lần, có thể do khác phần cứng và thêm lớp WSL2, nhưng mọi xu hướng đều khớp, kể cả việc `coalesce(4)` ra 3 file (Bảng 6). Chỗ khác là số task của AQE skew join, vì số partition sau khi tách phụ thuộc kích thước partition thực tế lúc chạy.

Bảng 6. So sánh với bảng kết quả tham chiếu của bài lab.

Chỉ số	Tham chiếu	Lần 2
E1, 200 partition, AQE off	1.29 s	2.20 s
E1, AQE on	0.21 s	0.53 s
E2, broadcast hint	0.24 s	0.40 s
E3 baseline (median / max task)	110 / 1,263 ms	165 / 2,366 ms
E3 salting	2.97 s	4.01 s
E3 AQE skew join (thời gian, task)	1.73 s, 43 task	2.22 s, 27 task
E4 coalesce(4)	1.85 s, 3 file	3.12 s, 3 file
E5 không cache / có cache	4.11 / 0.70 s	6.40 / 0.99 s

3 Trả lời câu hỏi checkpoint

3.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh?

Dữ liệu shuffle của E1 chưa tới 1.2 MiB (Mục 2.1). Chia ra 200 phần thì mỗi task reduce chỉ làm việc khoảng 6 ms, còn chi phí lập lịch, serialize và khởi tạo của 204 task lớn hơn phần việc thật, nên bản này chậm hơn bản 8 partition khoảng 6.7 lần (Bảng 4). Một partition chạy nhanh ở đây chỉ vì shuffle quá nhỏ. Với dữ liệu thật, một reducer phải ôm toàn bộ dữ liệu shuffle: không có song song, dễ spill ra đĩa và dễ OOM. Thực tế nên nhắm khoảng 100–200 MB mỗi partition shuffle, hoặc để AQE tự quyết.

3.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết?

Khi stage map của shuffle chạy xong, AQE có kích thước thật của từng partition shuffle (*map output statistics*). `AQEShuffleRead coalesced` cho một reducer đọc gộp nhiều partition nhỏ liền kề cho tới khi đạt `advisoryPartitionSizeInBytes`; trong E1, 200 partition chỉ còn một task reduce và tổng số task giảm từ 204 xuống 5 (Hình 2). Planner tĩnh chỉ có ước lượng dựa trên kích thước file, và ước lượng này hay sai sau filter, join hoặc aggregate.

3.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng?

Bằng chứng là chênh lệch giữa task max và median ở stage join của baseline: **2,366 ms so với 165 ms**, và task chậm nhất đọc 6,264,438 bản ghi trong khi median là 262,619 (Bảng 3, Hình 7).

Salting cắt được task max (Hình 8) nhưng thời gian tổng gần như không đổi. Một phần vì salting thêm việc ở mọi chỗ: `rand()` cho từng dòng, thêm cột join, nhân dòng dim lên 64 lần. Phần lớn hơn là khoảng 6 triệu dòng hot vẫn phải chia trên 16 partition chạy trên 4 core: tổng khối lượng việc không giảm, chỉ được chia lại, nên median tăng. Lần chạy đầu cũng vậy, salting chỉ nhanh hơn baseline một chút (4.28 s so với 4.62 s), còn bảng tham chiếu của đề thậm chí ghi salting chậm hơn (2.97 s so với 2.45 s). Salting sẽ thắng rõ khi có nhiều core và nhiều partition, vì lúc đó một straggler giữ hàng trăm executor rảnh; nó cũng có lợi khi mỗi dòng xử lý nặng (dòng rộng, join many-to-many, có spill) hoặc khi skew nặng hơn, ví dụ 90 % hot.

3.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu?

Một partition bị coi là lệch khi nó vừa lớn hơn `skewedPartitionFactor` lần median, vừa lớn hơn `skewedPartitionThresholdInBytes` [1]. Mặc định là 5 lần và 256 MB; bài lab hạ xuống 3 lần và 16 MB cho vừa dữ liệu nhỏ. Partition hot ở đây là 61.9 MiB so với median 3.4 MiB, thỏa cả hai điều kiện. AQE tách partition đó theo các khoảng map output (4 split, nên 16 partition thành 19), rồi nhân bản partition tương ứng ở phía còn lại; kế hoạch hiện `SortMergeJoin(skew=true)` (Hình 9), và số bản ghi lớn nhất mỗi task xấp xỉ một phần tư baseline (Bảng 3).

AQE có mấy giới hạn mà salting không có. Nó chỉ áp dụng cho join có shuffle ở cả hai phía (sort-merge hoặc shuffled hash join), và không tách được phía phải giữ nguyên của outer join. Ngưỡng phải hợp với dữ liệu, nếu không nó sẽ không kích hoạt, đúng như chúng tôi gặp ở E3. Quan trọng nhất, AQE không xử lý skew trong `groupBy` hay skew phát sinh sau join, trong khi salting hoặc aggregate hai pha làm được.

3.5 Khi nào không nên broadcast dù một phía "nhỏ"?

Không nên broadcast khi bảng vẫn lớn lúc nằm trong bộ nhớ, vì driver phải collect rồi gửi bản sao tới mọi executor, dễ OOM ở driver hoặc broadcast timeout. Cũng không nên khi thống kê đánh giá thấp kích thước thật (sau filter hoặc UDF), khi join là full outer join hoặc phía cần broadcast lại là phía được giữ của outer join, và khi join chạy lặp lại trên dữ liệu thay đổi nên mỗi lần phải dựng lại broadcast. Bảng zones trong E2 chỉ có 265 dòng và không rơi vào trường hợp nào ở trên, nên broadcast là đúng (Hình 3).

3.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước?

`repartition(n)` luôn shuffle, nhờ vậy partition cân bằng (4 file đều nhau) và stage trước giữ được mức song song, cái giá là thêm một shuffle. `coalesce(n)` gộp partition có sẵn qua phụ thuộc hẹp, không shuffle nên rẻ hơn, nhưng file ra có thể lệch: ở E4 chỉ có 3 file vì đầu vào chỉ có 3 row group. Cũng vì là phụ thuộc hẹp, `coalesce` kéo luôn stage trước về cùng số task [4]; `coalesce(1)` đọc và ghi trong một task duy nhất (Bảng 4).

3.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không?

DataFrame được cache trong E5 là kết quả của một bước đắt (`dropDuplicates`, shuffle toàn bộ khoảng 3 triệu dòng) và được dùng lại ba lần, nên cache giúp bước này chỉ chạy một lần thay vì ba (Mục 2.5, Hình 11). Một phép quét Parquet chỉ lấy vài cột vốn đã rẻ nhờ đọc vectorized, column pruning và predicate pushdown; cache nó chỉ tốn thêm bộ nhớ và thời gian materialize. Nguyên tắc là cache cái gì vừa đắt vừa được dùng lại, xong thì gọi `unpersist()`.

3.8 Nên tinh chỉnh gì trước khi thêm phần cứng?

1. Đọc kế hoạch và Spark UI: tìm stage chậm nhất, spill, thời gian GC, và tỉ lệ task max/median để phát hiện skew (như Bảng 3).
2. Lọc và chọn cột càng sớm càng tốt: partition pruning, predicate/column pushdown, dùng định dạng cột (Parquet/ORC).
3. Để partition shuffle có kích thước hợp lý: bật AQE (coalesce và skew join), nhắm khoảng 100–200 MB mỗi partition.
4. Chọn đúng kiểu join: broadcast bảng nhỏ; nếu có skew thì dùng AQE skew join, salting, hoặc tách hot key.
5. Tránh UDF Python (dùng hàm built-in hoặc pandas UDF), chỉ cache khi cần, và tránh sinh file nhỏ khi ghi (repartition hoặc compaction).

4 Khuyến nghị cho job production trên cluster 20 node

Mỗi khuyến nghị ứng với một thí nghiệm, đặt trong bối cảnh nhiều core hơn hẳn `local[4]`:

1. E1: bật AQE, đặt `shuffle.partitions` ban đầu khoảng 2–3 lần tổng số core (400–800) và để `coalescePartitions` gộp về khoảng 64–128 MB mỗi partition; không đặt cứng 1 hay 8.
2. E2: broadcast bảng dimension nhỏ, có thể nâng `autoBroadcastJoinThreshold` lên khoảng 50–100 MB nếu executor đủ bộ nhớ, dùng `broadcast()` khi thống kê sai.
3. E3: để AQE skew join chạy với ngưỡng mặc định; với hot key đã biết hoặc skew trong `groupBy`/outer join thì dùng salting hoặc tách hot key.
4. E4: ghi bằng `repartition(n)` hoặc `repartition(col)` để mỗi file khoảng 128 MB–1 GB; tránh `coalesce(1)` trên dữ liệu lớn.
5. E5: chỉ `persist(MEMORY_AND_DISK)` kết quả đắt được dùng lại ít nhất hai lần, rồi `unpersist()` ngay.

5 Kết luận

Cả năm thí nghiệm đã chạy hai lần, các phần mã tự viết (AQE trong E1, gợi ý broadcast trong E2, salting và AQE skew join trong E3) đều cho kế hoạch như đề yêu cầu. Ngay trên một máy, khác biệt đã đo được: số partition shuffle không hợp làm truy vấn chậm khoảng 6.7 lần và AQE tự sửa được; skew lộ rõ qua tỉ lệ task max/median khoảng 14 lần; AQE skew join giảm gần một nửa thời gian tổng mà không sửa truy vấn, còn salting chỉ chia lại việc khi số core có hạn; `coalesce` rẻ nhưng có thể sinh file lệch; cache chỉ có lợi khi kết quả đắt và được dùng lại.

Tài liệu tham khảo

[1] The Apache Software Foundation, “Performance Tuning – Spark SQL, DataFrames and Datasets,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[2] The Apache Software Foundation, “Monitoring and Instrumentation (REST API),” Apache Spark Documentation. https://spark.apache.org/docs/latest/monitoring.html

[3] M. Armbrust *et al.*, “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2015.

[4] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. Sebastopol, CA: O’Reilly Media, 2018.

Phụ lục: Trích đoạn mã nguồn

Các tệp nộp kèm được liệt kê ở Bảng 7.

Bảng 7. Danh mục tệp nộp kèm.

Tệp	Nội dung
`l06_experiments.py`	Script thí nghiệm đã hoàn thành
`results.csv`, `RESULTS.md`	34 dòng (2 lần × 17 biến thể); bảng lần chạy thứ hai và nhận xét
`01_run1.txt`, `02_run2.txt`, `03_e3_task_quantiles.txt`	Log hai lần chạy; phân vị task ba stage join E3

Ở E1, chúng tôi bật lại AQE nhưng giữ 200 partition để xem AQE tự gộp:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__shuffle__partitions=200)
run("E1", "AQE on,  shuffle.partitions=200", q(), "AQE coalesces small
 ↪  shuffle partitions")
```

Ở E2, ngưỡng được để ở -1 nhưng gắn gợi ý broadcast cho bảng zones:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__autoBroadcastJoinThreshold=-1)
run("E2", "AQE on,  threshold -1 + broadcast() hint",
    trips_df().join(F.broadcast(zones_df()), F.col("PULocationID") ==
 ↪  F.col("LocationID"))
    .groupBy("Borough").agg(F.count("*").alias("trips")),
    "an explicit hint beats the threshold")
```

Salting ở E3 chỉ gắn salt cho hot key; phía dim nhân dòng hot lên `SALT` = 64 bản:

```
hot = F.col("key") == 0
fact_s = fact.withColumn(
    "salt", F.when(hot, (F.rand(seed=5) *
     ↪  SALT).cast("int")).otherwise(F.lit(0)))
salts = spark.range(SALT).select(F.col("id").cast("int").alias("salt"))
dim_s = (dim.filter(~hot).withColumn("salt", F.lit(0))
         .unionByName(dim.filter(hot).crossJoin(salts)))
salted = (fact_s.join(dim_s, ["key", "salt"])
          .groupBy("segment").agg(F.sum("amount").alias("amount")))
run("E3", f"salted SMJ (salt={SALT}), AQE off", salted)
```

Biến thể cuối của E3 bật AQE skew join và hạ ngưỡng cho vừa dữ liệu cỡ lab, đồng thời tắt coalesce để thấy rõ phần tách:

```
set_conf(spark__sql__adaptive__enabled=True,
         spark__sql__adaptive__skewJoin__enabled=True,
         spark__sql__adaptive__coalescePartitions__enabled=False,
         spark__sql__adaptive__advisoryPartitionSizeInBytes="4m",
 ↪  spark__sql__adaptive__skewJoin__skewedPartitionThresholdInBytes="16m",
         spark__sql__adaptive__skewJoin__skewedPartitionFactor=3)
run("E3", "skewed SMJ, AQE skewJoin on",
    fact.join(dim, "key").groupBy("segment").agg(F.sum("amount")),
    "look for AQEShuffleRead (skewed) in the SQL tab")
```